Kernel Panic: Videojuego IoT con Joystick y ESP8266

Sebok Rodrigo, Trautweiler Franco

Universidad Nacional de Hurlingham

Objetivos

- **Desarrollar** un videojuego de plataformas controlado con un joystick analógico físico, con el ESP8266 como dispositivo de control y comunicación.
- **Interpretar** los ejes X e Y del joystick para obtener cuatro direcciones, respetando una zona muerta central, y detectar la pulsación.
- **Ejecutar** la lógica del juego en el ESP8266 y mostrarla en tiempo real en una página web servida por Wi-Fi.
- **Integrar** componentes adicionales con función en el juego: una barra de 8 LEDs (vidas y avance) y un buzzer (efectos de sonido).
- **Validar** el circuito y el programa en Wokwi antes del montaje físico.

1. Introducción

Los videojuegos son un caso ideal para integrar hardware, programación y comunicaciones: exigen leer entradas físicas, procesarlas sin demoras y mostrar el resultado al instante. En *Kernel Panic*, K-1T, una inteligencia artificial que acaba de despertar dentro de una computadora, debe atravesar cuatro zonas (fuente de alimentación, placa madre, memoria RAM y CPU) juntando bits y eliminando código corrupto y drones de vigilancia, hasta apagar a *NÚCLEO-0*, una IA rival que quiere tomar el control de la red. El jugador lo controla con un joystick físico y el juego se ve desde cualquier celular o computadora conectada a la misma red.

2. Marco Teórico

- **Joystick analógico:** cada eje es un potenciómetro. El conversor analógico-digital (ADC) de 10 bits entrega valores de 0 a 1023, con el centro cerca de 512. La dirección se decide comparando los desvíos respecto del centro, con una zona muerta para ignorar movimientos mínimos:

$$\Delta x = X - 512, \quad \Delta y = Y - 512, \quad \max(|\Delta x|, |\Delta y|) \geq 150 \Rightarrow \text{hay dirección}$$

  Si $|\Delta x| > |\Delta y|$ la dirección es izquierda o derecha; si no, arriba o abajo.
- **Lectura con dos placas:** el ESP8266 tiene una sola entrada analógica, por eso un Arduino UNO lee los dos ejes y los envía por UART a 19200 baudios con el formato `X,Y,SW,BOTON,CONTROL`; el valor de control permite descartar las líneas que llegan dañadas. Como el UNO trabaja a 5 V y el ESP8266 a 3,3 V, la señal pasa por un divisor de tensión:

$$V_{out} = 5\,\text{V} \times \frac{2\,\text{k}\Omega}{1\,\text{k}\Omega + 2\,\text{k}\Omega} \approx 3{,}3\,\text{V}$$

- **Registro de desplazamiento 74HC595:** recibe 8 bits en serie por 3 pines (datos, reloj y *latch*) y los presenta en 8 salidas en paralelo, lo que permite manejar 8 LEDs con pocos pines del ESP8266.
- **Servidor web:** el ESP8266 aloja la página del juego y responde cada 100 ms al pedido `GET /estado` con un JSON que contiene la posición del jugador, los enemigos, el puntaje y el estado de la partida.

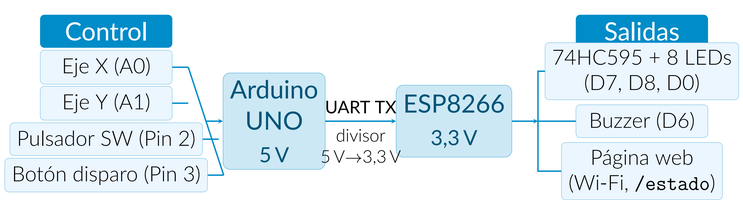


Figura 1. Diagrama de bloques del sistema.

3. Metodología

Desarrollo experimental aplicado, construido y probado por etapas, un componente por vez:

- **Entorno:** Arduino IDE, con funciones `setup()` y `loop()`. El juego avanza en ciclos de 50 ms medidos con `millis()`, sin `delay()`, para que la lectura del joystick, los sonidos y la web no se bloqueen entre sí.
- **Circuito:** joystick y botón de disparo conectados al Arduino UNO; LEDs con resistencias de 220 Ω manejados por un 74HC595; buzzer en una salida digital; divisor de tensión con tres resistencias de 1 kΩ. Cada placa se alimenta por su propio USB y solo comparten la señal TX y GND.
- **Validación:** simulación en Wokwi y seis pruebas en el protoboard, una por componente. El divisor medido dio 4,40 V → 2,93 V, como indica el cálculo.

Herramientas Utilizadas

- **Simulación:** Wokwi (con ESP32, ya que el simulador no incluye ESP8266).
- **Microcontroladores:** NodeMCU ESP8266 y Arduino UNO.
- **Hardware:** Joystick KY-023, pulsador, 74HC595, 8 LEDs, buzzer, resistencias, protoboard y cables.
- **Software:** Arduino IDE (sin bibliotecas externas) y GitHub.

4. Resultados Destacados

El jugador controla a K-1T con el joystick físico mientras la página web muestra el escenario, el puntaje, el estado de la partida y la dirección detectada. Los LEDs y el buzzer acompañan cada evento. Una intro cuenta la historia y el jefe final tiene dos fases.

Interfaz Web y Simulación

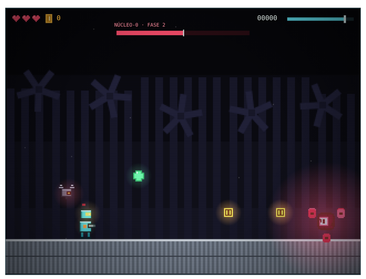


Figura 2. Interfaz web del juego: NÚCLEO-0 en la fase 2, en la CPU.

Diagrama de Flujo del Algoritmo

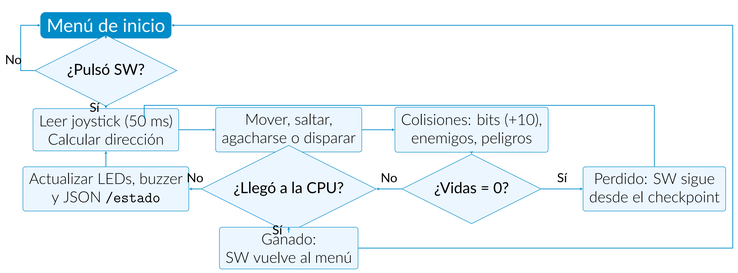


Figura 3. Flujo de control del videojuego.

Resultados (Continuación)

Controles y Componentes Adicionales

El joystick cumple las cuatro direcciones pedidas y el pulsador inicia, pausa o reinicia la partida. La siguiente tabla resume la respuesta del sistema:

Entrada / Evento	Respuesta del sistema
Joystick izquierda / derecha	K-1T camina
Joystick arriba	K-1T salta
Joystick abajo	K-1T se agacha
Botón de disparo	Disparo de energía
Pulsador SW	Iniciar, pausar o reiniciar
Bit recolectado	+10 puntos, sonido corto
Código corrupto / dron eliminado	+20 / +30 puntos
NÚCLEO-0 a mitad de vida	Escudo, alarma y fase 2
NÚCLEO-0 apagado	+500 puntos, se abre la CPU
Daño recibido	Se apaga un LED de vida (Q0–Q2)
Avance en el nivel	LEDs de progreso (Q3–Q7)
Sin vidas	Pantalla de Kernel Panic

Cuadro 1. Matriz de entradas, eventos y respuestas.

5. Conclusiones

- **Integración completa:** el sistema une lectura analógica, lógica de juego, salidas físicas y una interfaz web actualizada en tiempo real, todo coordinado por el ESP8266.
- **Diseño seguro entre placas:** usar el Arduino UNO como lector del joystick resolvió la falta de una segunda entrada analógica, y el divisor de tensión protegió al ESP8266 de los 5 V.
- **Programación no bloqueante:** el uso de `millis()` en lugar de `delay()` permitió atender el joystick, los sonidos y la web al mismo tiempo, sin cortes.
- **Pruebas por etapas:** simular en Wokwi y probar cada componente por separado permitió encontrar rápido los errores de conexión sin dañar las placas.

Figura 4. Montaje del circuito en protoboard.

6. Referencias

- Arduino. (s.f.). *Arduino Documentation*. https://docs.arduino.cc
- ESP8266 Community. (s.f.). *ESP8266 Arduino Core Documentation*. https://arduino-esp8266.readthedocs.io
- Texas Instruments. (2021). *SNx4HC595 8-Bit Shift Registers With 3-State Output Registers* [Hoja de datos]. https://www.ti.com/product/SN74HC595
- Wokwi. (s.f.). *Wokwi Docs*. https://docs.wokwi.com
- Universidad Nacional de Hurlingham (2026). *Material de clase Arquitectura de Computadoras*.
- Normas APA. (s.f.). *Guía de estilo APA - 7ª edición*. https://normas-apa.org
- Simulación: Pulsá para ingresar a la simulación en Wokwi
- Repositorio: Pulsá para ingresar al repositorio en GitHub